Prove Project- Joaquin Bejarano

RF Emergency Shutoff

This is my proposed code for the car initiated RF heartbeat and the request to shut off high voltage through the repository's existing contactor control path. The new code still needs to be integrated with its radio, CAN, startup, and shutdown software and reviewed electrical team. I decided to wait until the braking actuators are specified before writing their output code. Brake actuator output is not included here.

Which files are which?

`code/rf/` holds the shared, hardware-independent rules:

Files	**What they do**
rf_guard.c and .h	Car-side timeout, fresh response checks, fault latch, and reset rules
rf_station.c and .h	Station response, Allow/STOP state, and laptop permission lease
rf_nodes.c and .h	Proposed CAN check-in between the car and important nodes
rf_endpoint.c and .h	Proposed node-side stop logic; needs actual hardware callbacks

`code/esp/` connects those rules to ESP32 software:

Files	**What they do**
RfCar.cpp and RfCar.h	Car-side radio/watchdog tasks and CAN response handling
RfStationApp.cpp and RfStationApp.h	Station-side radio task and USB control/status interface
RfRadio.cpp and RfRadio.h	Shared ESP32 adapter to the existing LoRa driver
RfReviewConfig.h	Proposed radio settings; CAN identities and hardware approval are intentionally unconfigured
RfNoBulk.cpp	Makes the proposed heartbeat-only radio mode reject regular bulk telemetry
contactorControl.c	Proposed replacement of an existing repository file, preserving its contactor-off destinations
pecanTrySend.c	Proposed nonblocking CAN send helper for the ESP32

The filenames above are grouped by role, not by which deliverable they belong to: both the heartbeat and shutdown request need shared and ESP-specific pieces. The car and station must use compatible radio settings. This folder alone will not compile as a standalone application: the ESP files depend on the existing repository, and its startup, build, driver, and CAN entry points still need approved changes.

Before integration, we need to confirm the meaning of the 60 ms target (spacing between requests versus full round trip), the RF settings, CAN IDs, the physical contactor-off response, and the reset policy.

Where to start reading the code
1. rf_guard.c: follow rf_make_request(), rf_receive(), then rf_tick().
2. rf_reset(): see why receiving a good answer doesn't automatically restart.
3. rf_station.c: see how a fresh answer is made and how STOP works.
4. RfCar.cpp: see how those functions are proposed to run in separate ESP tasks.
5. contactorControl.c: see how the existing off destinations would be reused.
The files are separated because the radio, shutdown, and CAN have different jobs. The basic heartbeat logic itself stays small and can be tested without an ESP board.

How this works in plain English

The main idea

Think of the car repeatedly asking the station, “Are you still there, and am I still allowed to operate?” The station answers that specific question. The car doesn't accept any random radio traffic as proof that everything is okay. Each question has a session number and a changing request number. The session changes when the car starts again. The request number changes every time it asks. The station sends those same numbers back, so the car can tell which question the answer belongs to. An old answer can't keep the car running forever.

The clock and the latch

The car remembers when it last received a good positive answer. A separate task checks that clock without waiting for the radio. If more than 200 milliseconds pass, it records a fault. A valid “stop” answer also records a fault. The fault is a latch: once it turns on, it stays on. Hearing the station again is not enough to clear it. That's important because we don't want the system suddenly enabling itself when somebody reconnects a cable or comes back into radio range. The reset rules in this version are a proposal for the lead to approve. The station must be allowing replies, the car must have several fresh answers, the important nodes must be ready, the outputs must be confirmed off, and someone must explicitly request a local reset. Even then, reset does not turn the contactors on.

How shutting off is requested

I followed the existing contactor-off command instead of choosing a new pin. That path sends CAN disable commands to the precharge and power distribution nodes. CAN is the wired network connecting the vehicle's controllers. The proposed contractor manager gives off requests on their own latch, so they don't have to wait behind a queue of normal commands. It also blocks enable requests while RF permission is missing and periodically repeats the off request. But sending a command is not the same as opening a physical contactor. The code audit found that one receiver currently only logs the disable command, and the precharge receiver can spend a long time waiting for voltage measurements. Those parts need changes and physical verification from the team.

Why the other nodes also need protection

If the car's radio controller freezes or the CAN cable fails, a command from that controller might never arrive. The important nodes therefore need their own missing-message timers and local stop behavior. The package includes portable logic for that, but the board owners must connect it to their real outputs. The backup power board is different. It shouldn't cut power to the whole car as the first response, because that can also turn off the telemetry and other useful equipment. The team must decide how it verifies that the primary shutdown failed before escalating.